1.Aim of the practical: Write a program based on Artificial neural (using Tensor flow). Also specify the data set used with number of input and output variables.

2.Objective: The objective of this program is to build a simple Artificial Neural Network (ANN) using TensorFlow to predict outputs based on given input features.

- The input features are normalized and the outputs are one-hot encoded to train the network effectively.
- The network uses forward propagation and backpropagation automatically through TensorFlow to learn from the data.

3.Algorithm/Flowchart:

```
from sklearn.datasets import make_circles,make_moons,make_blobs
import numpy as np
data, target = make_circles(n_samples=1000, noise=0.03)
data
target
import matplotlib.pyplot as plt
plt.scatter(data[:,0],data[:,1],c=target)

import tensorflow as tf
from tensorflow.keras import Sequential
from tensorflow.keras.layers import Input, Dense
x_train, x_test, y_train, y_test = train_test_split(data, target, test_size=0.25, random_state=42)

model = Sequential([
    Input(shape=(2,)),
    Dense(4, activation='relu'),
    Dense(1, activation='sigmoid')
])
```

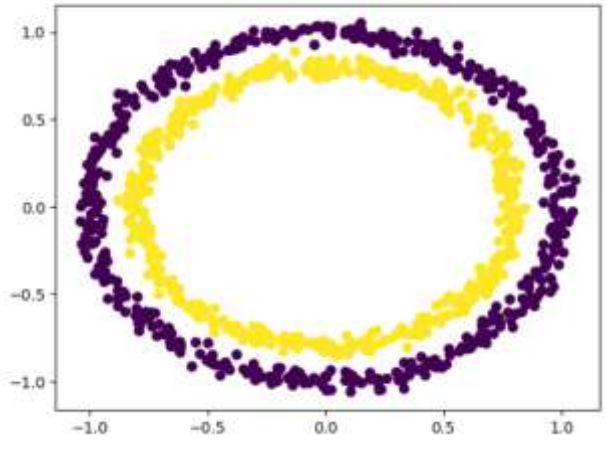
```
model.summary()

loss = tf.keras.losses.BinaryCrossentropy()
optimizer = tf.keras.optimizers.SGD(learning_rate=0.1)

model.compile(optimizer=optimizer, loss=loss, metrics=['accuracy'])
model.fit(x_train, y_train, epochs=5, validation_data=(x_test, y_test))
```

4.Result/Output:

```
array([1, 1, 1, 1, 0, 1, 1, 1, 0, 1, 0, 1, 0, 1, 1, 1, 0, 0, 0, 0, 0, 1,
       1, 1, 1, 0, 0, 1, 1, 0, 0, 0, 1, 0, 1, 1, 1, 1, 1, 1, 0, 1, 1, 1,
       1, 1, 1, 1, 0, 0, 1, 0, 0, 0, 1, 0, 1, 1, 0, 0, 1, 1, 1, 1, 1, 1,
       0, 1, 1, 0, 0, 0, 0, 0, 1, 1, 0, 0, 0, 1, 1, 0, 1, 1, 0, 0, 1, 0,
       0, 1, 1, 1, 1, 1, 0, 1, 0, 0, 1, 0, 1, 1, 0, 1, 0, 1, 0, 0, 1, 1,
       1, 1, 0, 0, 1, 0, 1, 1, 0, 0, 0, 1, 0, 0, 1, 0, 0, 1, 1, 1, 1, 0,
       0, 0, 0, 1, 1, 0, 0, 1, 1, 1, 0, 1, 0, 0, 1, 0, 1, 1, 1, 0, 1, 0,
       0, 0, 0, 1, 1, 1, 0, 0, 0, 0, 1, 0, 0, 0, 1, 1, 0, 1, 0, 0, 1,
       1, 1, 0, 0, 0, 1, 1, 0, 0, 0, 1, 1, 0, 0, 1, 1, 1, 0, 0, 1, 0, 0,
       1, 0, 0, 1, 1, 1, 1, 0, 1, 1, 1, 0, 0, 0, 1, 0, 1, 0, 0, 1, 1, 0,
       1, 1, 0, 0, 1, 0, 1, 0, 0, 1, 0, 0, 1, 0, 0, 0, 1, 1, 0, 1, 1, 1,
       0, 0, 1, 0, 1, 1, 0, 0, 0, 0, 1, 0, 1, 1, 0, 0, 1, 0, 1, 0, 0, 1,
       0, 0, 1, 0, 0, 0, 0, 1, 0, 1, 0, 1, 0, 0, 1, 1, 1, 1, 1, 1, 0, 1,
       0, 1, 0, 0, 0, 0, 1, 0, 1, 0, 1, 1, 0, 0, 0, 0, 0, 1, 0, 0, 1, 1,
       1, 0, 0, 1, 0, 1, 0, 1, 1, 1, 1, 1, 1, 0, 0, 1, 0, 1, 1, 1, 0, 1,
       0, 0, 1, 1, 1, 1, 1, 1, 0, 1, 0, 0, 1, 1, 1, 1, 0, 1, 0, 1, 1, 1,
       1, 0, 1, 0, 1, 1, 1, 1, 0, 1, 0, 0, 1, 1, 0, 0, 1, 1, 1, 1, 0, 1,
```

```
array([[-0.64856294,  0.44738364],
       [ 0.41889701, -0.6405426 ],
       [-0.7492267 ,  0.09080134],
       ...,
       [ 0.41324717, -0.65230896],
       [ 0.66265551,  0.50952191],
       [-0.80412554, -0.62465245]])
```

```
<matplotlib.collections.PathCollection at 0x14c0d8d3620>
```




```
Model: "sequential_8"
```

Layer (type)	Output Shape	Param #
dense_16 (Dense)	(None, 4)	12
dense_17 (Dense)	(None, 1)	5

```
Total params: 17 (68.00 B)

Trainable params: 17 (68.00 B)

Non-trainable params: 0 (0.00 B)
```

```
Epoch 1/5
24/24 ━━━━━━━━━━━━━━━━━━━━ 1s 13ms/step - accuracy: 0.5053 - loss: 0.6893 - val_accuracy: 0.5160 - val_loss: 0.6866
Epoch 2/5
24/24 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.5533 - loss: 0.6860 - val_accuracy: 0.5160 - val_loss: 0.6863
Epoch 3/5
24/24 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.5480 - loss: 0.6841 - val_accuracy: 0.5760 - val_loss: 0.6856
Epoch 4/5
24/24 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.5800 - loss: 0.6827 - val_accuracy: 0.5240 - val_loss: 0.6849
Epoch 5/5
24/24 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.5733 - loss: 0.6812 - val_accuracy: 0.5160 - val_loss: 0.6843
<keras.src.callbacks.history.History at 0x14c1de3b9d0>
```

Learning outcomes (What I have learnt):

- Learn to build and train an Artificial Neural Network (ANN) using TensorFlow/Keras.
- Understand how to preprocess data, including normalization of inputs and one-hot encoding of outputs.
- Gain knowledge on how the network learns via forward and backward propagation to make predictions.